

Sécurisation d'une API Gateway avec Keycloak

Préparée par Ons Ben Amor



Plan de la présentation

01

Keycloak : définition, services et protocoles

02

Intégration : Front, Gateway ou Microservices ?

03

Mots-clés : realm, client, token, issuer, JWKS...

04

Atelier Pratique

05

Configuration Keycloak et tests Postman

C'est quoi Keycloak ?

Une définition simple avant les détails techniques.

Définition

Keycloak est une solution open source de gestion des identités et des accès (IAM). Il centralise la connexion, les utilisateurs, les rôles et les tokens.

Idée principale

Au lieu de coder la sécurité dans chaque application, on délègue l'authentification et une partie de l'autorisation à Keycloak.



Utilisateur

Keycloak

API Gateway

Microservices

login → token → accès API

Les services fournis par Keycloak

Ce que Keycloak apporte à l'application.

Users

Gestion des utilisateurs, mots de passe, profils

Roles

Permissions globales ou par client

SSO

Connexion unique entre plusieurs applications

Tokens

Access token, refresh token, ID token

Clients

Déclaration des applications connectées

Identity providers

Connexion via Google, LDAP, SAML, etc.

Protocoles utilisés par Keycloak

Les standards qui permettent l'intégration avec Spring Boot et les applications web.

OAuth 2.0

Autorisation

Permet d'obtenir un access token pour accéder aux APIs.

OpenID Connect

Authentification + identité

Ajoute l'identité de l'utilisateur au-dessus de OAuth2.

SAML 2.0

SSO entreprise

Souvent utilisé dans les organisations et anciens systèmes.

JWT

Format du token

Token signé qui contient des claims : issuer, expiration, rôles...

Pourquoi Keycloak dans une architecture microservices ?

Le problème : beaucoup de services, une sécurité à garder cohérente.

Sans Keycloak

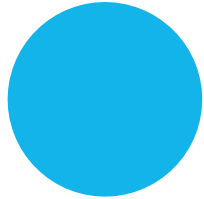
- Chaque service peut réinventer le login
- Risque de duplication du code de sécurité
- Gestion difficile des utilisateurs et rôles
- Changement de règle = plusieurs services à modifier

Avec Keycloak

- Sécurité centralisée
- Un token standard pour les appels API
- Gestion des rôles dans un seul endroit
- Ajout de services plus simple

Où intégrer Keycloak ?

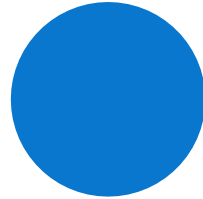
Front, Gateway et Microservices n'ont pas le même rôle.



Frontend

Redirection login et
récupération du token

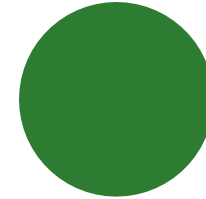
Expérience utilisateur



API Gateway

Validation du JWT avant le
routage

Protection globale



Microservices

Vérification fine des rôles

Sécurité métier

Recommandation : Front pour le login + Gateway pour filtrer + Microservices pour les règles sensibles.

Tableau comparatif

Le choix d'architecture expliqué clairement.

Niveau	Rôle	Avantage	Limite
Frontend	Login utilisateur et récupération token	UX claire	Peut être contourné
API Gateway	Valider JWT avant routage	Point d'entrée unique	Pas toute la logique métier
Microservice	Contrôler rôles/permissions	Sécurité fine	Configuration répétée
Meilleur choix	Combiner les niveaux	Équilibre sécurité / maintenabilité	Demande organisation

Dans l'atelier, nous sécurisons l'API Gateway, car elle est le point d'entrée des requêtes.

Mots-clés de Keycloak

Le vocabulaire nécessaire avant l'atelier.

Realm

Espace isolé de sécurité

Client

Application déclarée dans Keycloak

User

Utilisateur connecté

Role

Permission ou profil d'accès

Token

Preuve d'authentification

Client Secret

Mot de passe technique du client

Service Account

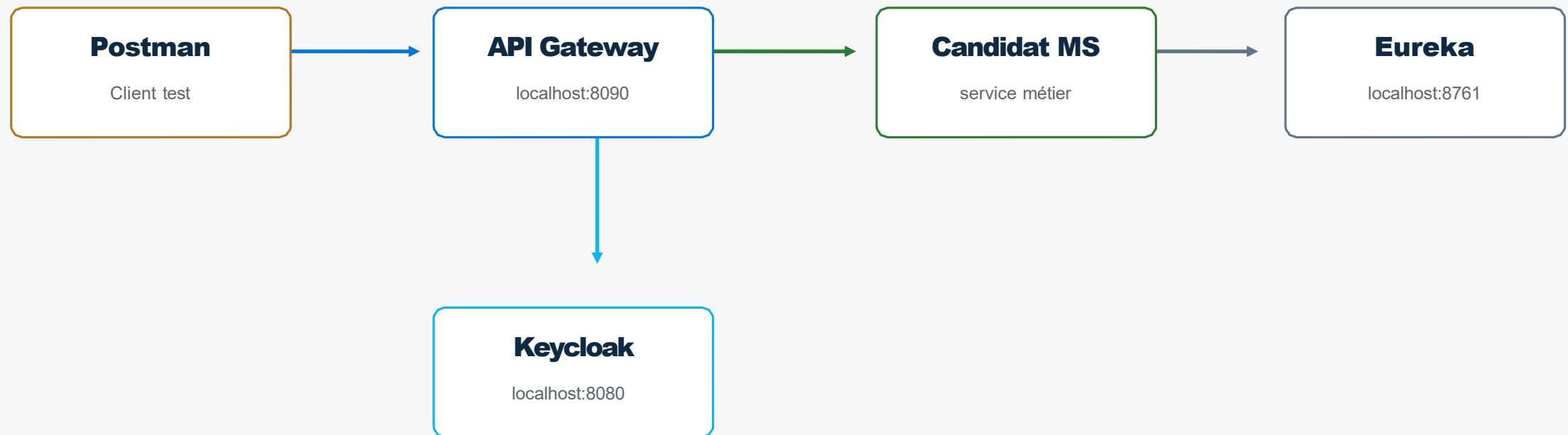
Compte technique pour Client Credentials

Issuer / JWKS

Émetteur du token + clés de vérification

Architecture de l'atelier

Ce que nous allons construire et tester.



Flux : Postman demande un token à Keycloak, puis appelle la Gateway avec Authorization: Bearer <token>.

ATELIER PRATIQUE

ÉTAPE 1

Ajouter Keycloak dans docker-compose.yml

```
keycloak:
  image: quay.io/keycloak/keycloak:26.3.1
  container_name: keycloak-server
  ports:
    - "8080:8080"
  environment:
    - KC_BOOTSTRAP_ADMIN_USERNAME=admin
    - KC_BOOTSTRAP_ADMIN_PASSWORD=admin
  command: start-dev
  volumes:
    - keycloak_data:/opt/keycloak/data
  restart: unless-stopped
```

Pourquoi cette partie ?

On ajoute Keycloak comme service Docker. Le navigateur accède à Keycloak via localhost:8080. Le mode start-dev est adapté à l'atelier.

À retenir

On ne lance plus Keycloak avec une installation locale. Docker Compose démarre Keycloak avec le reste de l'architecture.

Compte admin

admin / admin est créé au premier lancement, si le volume ne contient pas déjà une ancienne configuration.

ÉTAPE 2

Ajouter le volume Keycloak

```
volumes:  
  keycloak_data:  
  mysql_data:
```

Pourquoi keycloak_data ?

- Sauvegarde les realms créés dans Keycloak
- Sauvegarde les clients, secrets, rôles et utilisateurs
- Évite de tout refaire après un redémarrage
- Attention : docker compose down -v supprime les volumes

Règle simple

down garde la data • down -v supprime les volumes

ÉTAPE 3

Configurer la Gateway dans docker-compose.yml

```
api-gateway:
  build: ./gateway
  image: ons/apigateway:1.0
  ports:
    - "8090:8080"
  depends_on:
    - eureka-server
    - keycloak
  environment:
    - EUREKA_CLIENT_SERVICEURL_DEFAULTZONE=http://eureka-server:8761/eureka
    - SPRING_SECURITY_OAUTH2_RESOURCESERVER_JWT_ISSUER_URI=http://localhost:8080/realms/JobBoardKeycloak
    - SPRING_SECURITY_OAUTH2_RESOURCESERVER_JWT_JWK_SET_URI=http://keycloak:8080/realms/JobBoardKeycloak/protocol/openid-connect/certs
```

Ports

8080 est le port interne de la Gateway. On l'expose sur 8090 pour éviter le conflit avec Keycloak.

depends_on

La Gateway dépend de Keycloak et Eureka. Elle doit pouvoir communiquer avec eux.

JWT config

La Gateway devient un Resource Server : elle valide les JWT avant de router.

ÉTAPE 4

Comprendre issuer-uri et jwk-set-uri

issuer-uri

Répond à la question : qui a créé le token ?
Dans notre cas, le token obtenu par Postman contient l'émetteur localhost:8080.

jwk-set-uri

Répond à la question : où la Gateway récupère la clé publique pour vérifier le token ?
Dans Docker, la Gateway appelle Keycloak avec keycloak:8080.

Deux adresses pour deux contextes

Postman / Browser → localhost:8080
Gateway container → keycloak:8080

```
issuer-uri:  
http://localhost:8080/realms/JobBoardKeycloak  
  
jwk-set-uri:  
http://keycloak:8080/realms/JobBoardKeycloak/protocol/openid-connect/certs
```

ÉTAPE 5

Lancer l'architecture avec Docker Compose

```
# Dans le dossier du projet
docker compose up -d --build

# Vérifier les conteneurs
docker ps

# Voir les logs Keycloak si nécessaire
docker logs -f keycloak-server
```

Résultat attendu

Les conteneurs Keycloak, Eureka, MySQL, Gateway, Candidat et Job doivent être en statut Running.

URLs utiles

Keycloak : <http://localhost:8080>
Gateway : <http://localhost:8090>
Eureka : <http://localhost:8761>

Si problème de port

Arrêter l'ancien conteneur ou vérifier Docker Desktop > Containers.

ÉTAPE 6

Ouvrir l'interface Keycloak

URL

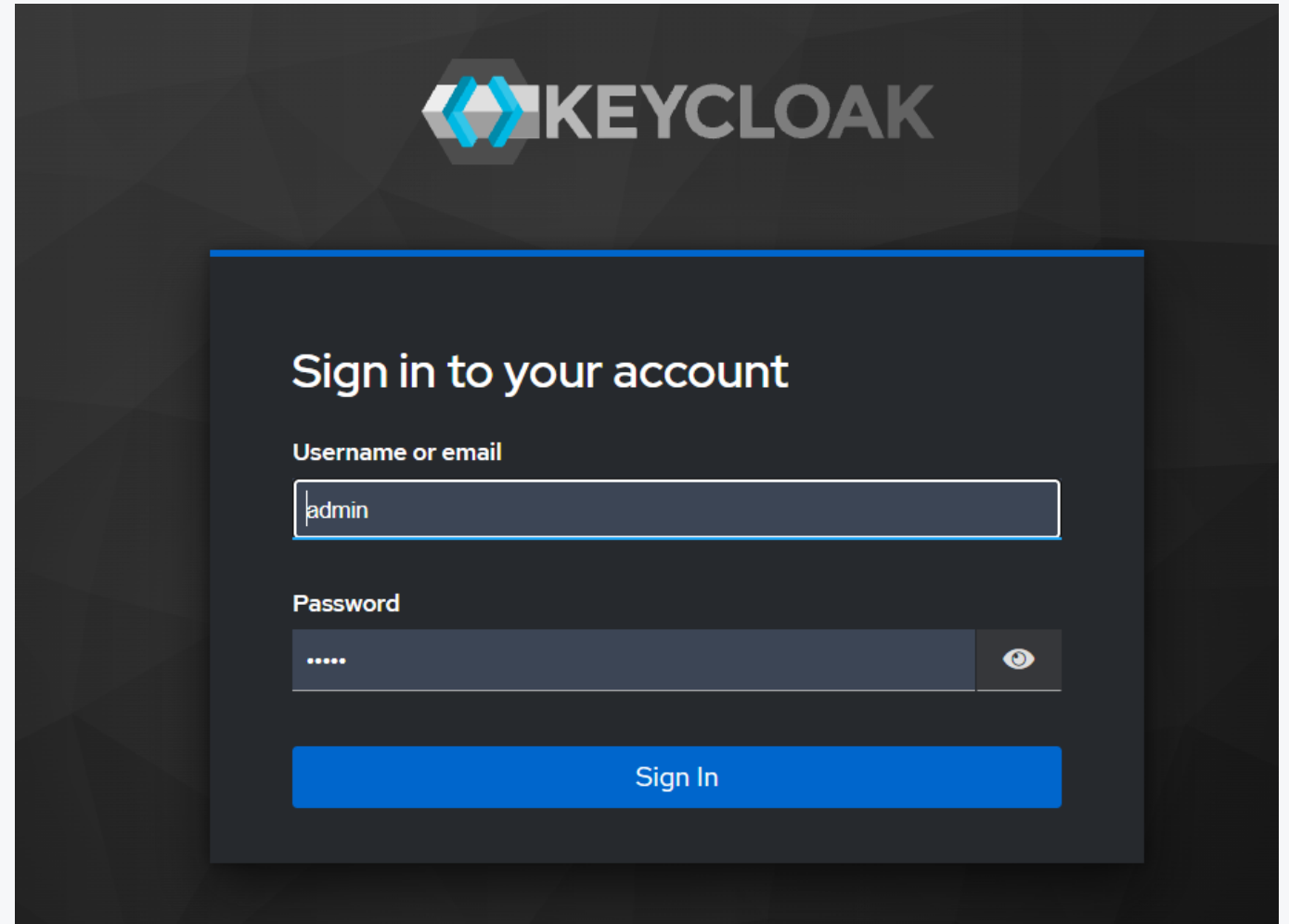
<http://localhost:8080>

Administration Console

Username : admin
Password : admin

Important

Si admin/admin ne fonctionne pas, c'est souvent une ancienne data dans le volume.



ÉTAPE 7

Créer le Realm JobBoardKeycloak

Action

Manage realms → Create realm → Realm name : JobBoardKeycloak
→ Create

Pourquoi un realm ?

Un realm est un espace isolé qui contient ses utilisateurs, ses clients, ses rôles et sa configuration.

À retenir

Chaque application ou environnement peut avoir son propre realm.

Keycloak Admin Console

1. Manage realms
2. Create realm
3. Name: JobBoardKeycloak
4. Create

ÉTAPE 8

Créer le client apiGateway

Action

Clients → Create client → Client type : OpenID Connect → Client ID : apiGateway

Pourquoi un client ?

Dans Keycloak, un client représente une application qui utilise Keycloak. Ici, notre Gateway est représentée par le client apiGateway.

Attention

Le Client ID est sensible à la casse : apiGateway ≠ ^gateway.

Clients > Create client

Create client

Clients are applications and services that can request authentication of a user.

- 1 General settings
- 2 Capability config
- 3 Login settings

Client type ⓘ OpenID Connect

Client ID * ⓘ apiGateway

Name ⓘ

Description ⓘ

Always display in UI ⓘ ☐ Off

ÉTAPE 9

Configurer les URLs et Capability config

Root URL: <http://localhost:8090>
Home URL: <http://localhost:8090>
Valid Redirect URI: http://localhost:8090/*
Web Origins: <http://localhost:8090>

Client authentication: ON
Service accounts roles: ON

Client authentication: ON:

Le client doit prouver son identité avec un **Client Secret**.

Service accounts roles : ON :

Le client peut demander un token **sans utilisateur**, comme une application technique.

Standard flow : ON

Il permet le login normal avec redirection vers Keycloak.

The screenshot displays the Keycloak configuration interface for a client. It is divided into two main sections: 'Access settings' and 'Capability config'.

Access settings:

- Root URL:** <http://localhost:8090>
- Home URL:** <http://localhost:8090>
- Valid redirect URIs:** http://localhost:8090/*
+ Add valid redirect URIs
- Valid post logout redirect URIs:** + Add valid post logout redirect URIs
- Web origins:** <http://localhost:8090>
+ Add web origins
- Admin URL:** <http://localhost:8090>

Capability config:

- Client authentication:** ☒ On
- Authorization:** ☐ Off
- Authentication flow:** ☒ Standard flow ☐ Implicit flow
- Direct access grants:** ☐ Direct access grants
- Service accounts roles:** ☒ Service accounts roles

ÉTAPE 10

Ajouter les dépendances dans le Gateway

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

webflux

Requis avec Spring Cloud Gateway, car la Gateway est réactive.

oauth2-resource-server

Permet de valider les JWT envoyés par Keycloak.

starter-security

Active les filtres de sécurité Spring Security.

ÉTAPE 11

Créer SecurityConfig.java

```

@Configuration
@EnableWebFluxSecurity
public class SecurityConfig {

    @Bean
    public SecurityWebFilterChain securityWebFilterChain(
        ServerHttpSecurity serverHttpSecurity) {

        return serverHttpSecurity
            .csrf(ServerHttpSecurity.CsrfSpec::disable)
            .authorizeExchange(exchange -> exchange
                .pathMatchers("/eureka/**").permitAll()
                .anyExchange().authenticated())
            .oauth2ResourceServer(oauth2 -> oauth2.jwt(jwt -> {}))
            .build();
    }
}

```

/eureka/ permitAll**

Eureka reste accessible pour l'enregistrement des services.

anyExchange authenticated

Toutes les autres routes exigent un token valide.

oauth2ResourceServer.jwt

La Gateway vérifie la signature du JWT avec les clés de Keycloak.

ÉTAPE 12

Tester sans token : 401 Unauthorized

GET <http://localhost:8090/candidats>
Authorization: aucun token

Résultat attendu

401 Unauthorized

Interprétation

La Gateway bloque la requête avant d'atteindre le microservice Candidat.

The screenshot shows a REST client interface with the following details:

- URL: `http://localhost:8090/mic1/candidats`
- Method: `GET`
- Response Status: `401 Unauthorized` (highlighted in red)
- Response Time: `403 ms`
- Response Size: `274 B`
- Response Headers: `Pass the correct auth credentials` (visible in the raw response view)

Key	Value	Description
Key	Value	Description

ÉTAPE 13

Récupérer le Client Secret

Action

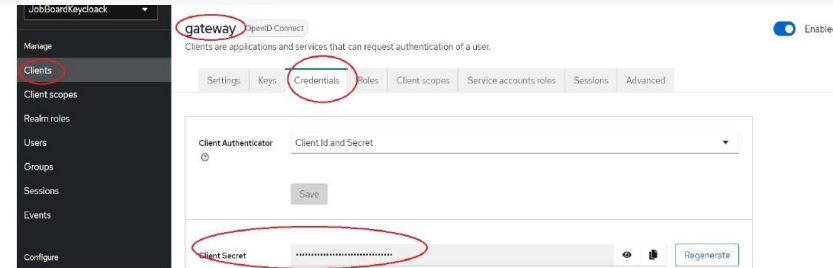
Client apiGateway → Onglet Credentials → Copier le Client Secret

Pourquoi ?

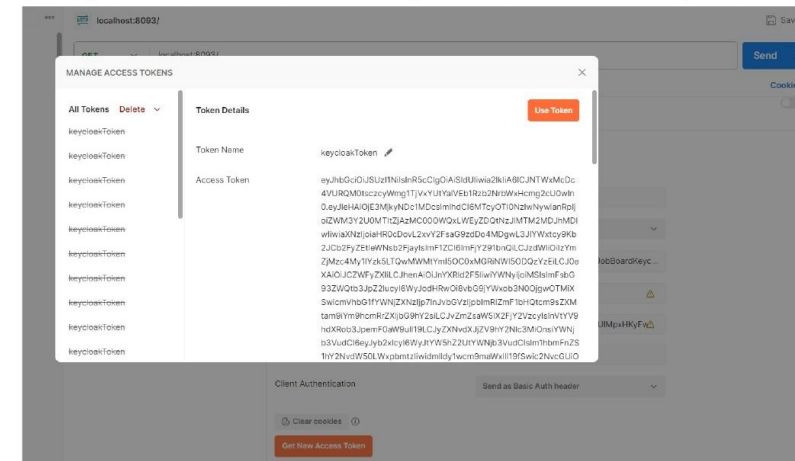
Le Client Secret est utilisé avec le Client ID pour demander un token avec le grant type Client Credentials.

Sécurité

Ne partage pas le secret dans un vrai projet. Pour l'atelier, il sert uniquement à la démonstration.

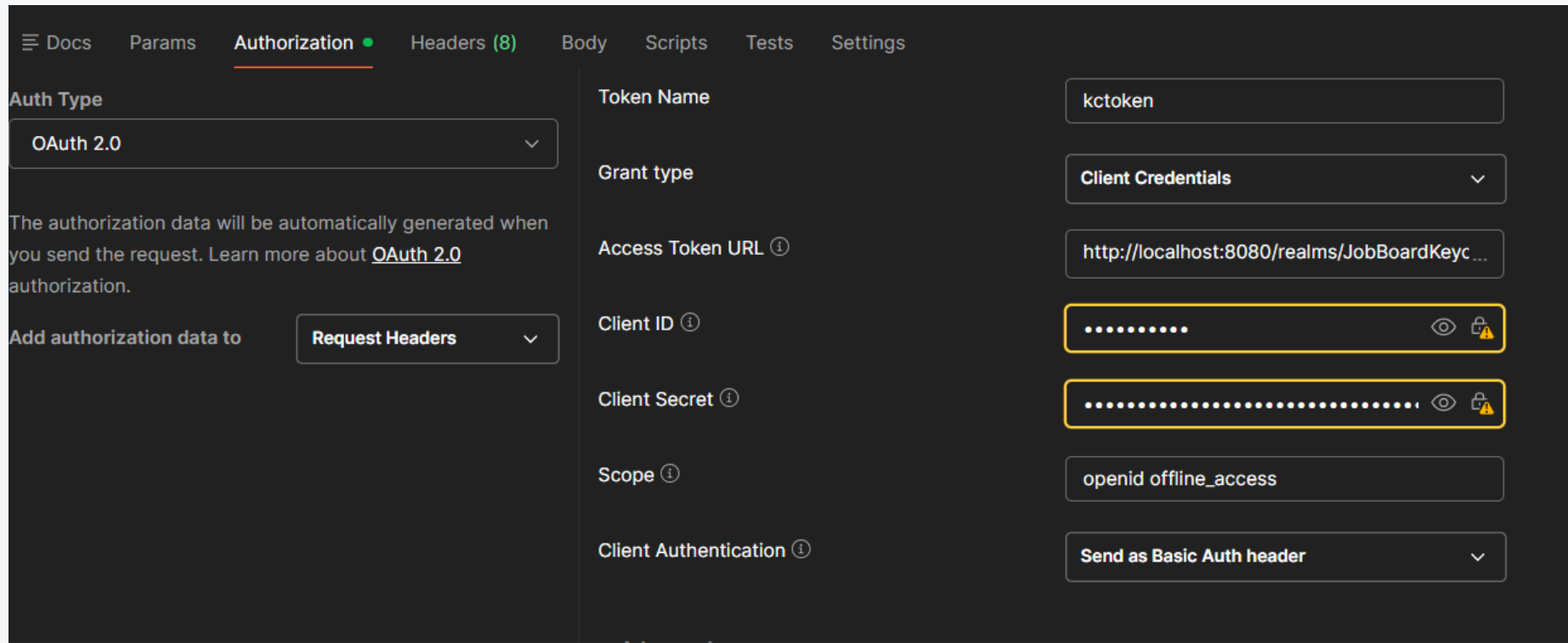


- **Scope** : openid offline_access (pour obtenir à la fois : un **ID Token** pour prouver l'identité de l'utilisateur et un **Refresh Token** pour un accès continu même après la fin de la session utilisateur)
- **Client Authentication** : Send a Basic Auth header
- Cliquer sur Get New Access Token. Vous pouvez voir la Valeur de token générée.



ÉTAPE 14

Générer un token avec Postman



The screenshot shows the Postman interface with the 'Authorization' tab selected. The 'Auth Type' is set to 'OAuth 2.0'. The 'Token Name' is 'kctoken'. The 'Grant type' is 'Client Credentials'. The 'Access Token URL' is 'http://localhost:8080/realms/JobBoardKeyc...'. The 'Client ID' and 'Client Secret' fields are masked with dots and highlighted with yellow boxes. The 'Scope' is 'openid offline_access'. The 'Client Authentication' is set to 'Send as Basic Auth header'.

Docs Params **Authorization** Headers (8) Body Scripts Tests Settings

Auth Type
OAuth 2.0

The authorization data will be automatically generated when you send the request. Learn more about [OAuth 2.0](#) authorization.

Add authorization data to Request Headers

Token Name
kctoken

Grant type
Client Credentials

Access Token URL ⓘ
http://localhost:8080/realms/JobBoardKeyc...

Client ID ⓘ
.....

Client Secret ⓘ
.....

Scope ⓘ
openid offline_access

Client Authentication ⓘ
Send as Basic Auth header

Pourquoi Client Credentials ?

Dans cet atelier, Postman agit comme un client technique qui demande un access token à Keycloak.

ÉTAPE 15

Tester avec token : 200 OK

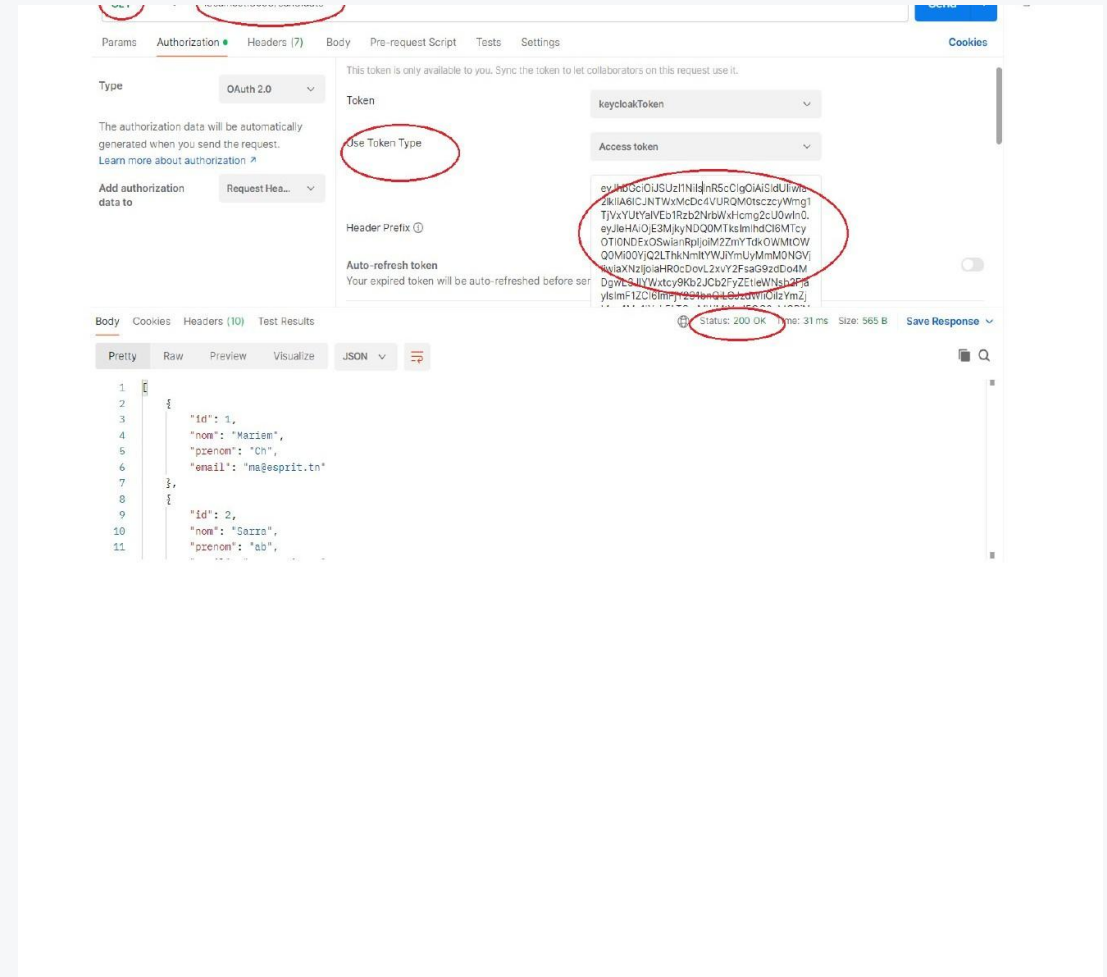
GET <http://localhost:8090/candidats>
Authorization: Bearer <access_token>

Résultat attendu

200 OK

Interprétation

La Gateway valide le token, puis route la requête vers le microservice Candidat.



Problèmes fréquents et corrections

Une slide de secours pour la démonstration en classe.

Client not enabled

Activer Service accounts roles dans le client Keycloak

401 avec token

Vérifier issuer-uri, jwk-set-uri et header Bearer

admin/admin refusé

Ancienne data dans le volume : reset ou récupérer admin

Port 8080 occupé

Changer le port ou arrêter l'ancien conteneur

Gateway ne démarre pas

Rebuild : `docker compose up -d --build`

Token expiré

Cliquer Refresh ou générer un nouveau token

Récapitulatif de l'atelier

Ce que nous avons réalisé.

- 1 Ajout de Keycloak dans Docker Compose
- 2 Ajout du volume keycloak_data
- 3 Configuration JWT de la Gateway
- 4 Création du realm JobBoardKeycloak
- 5 Création du client apiGateway
- 6 Génération du token dans Postman
- 7 Test 401 sans token puis 200 avec token

Conclusion

Keycloak centralise l'identité.

La Gateway protège le point d'entrée.

Les microservices gardent les règles métier fines.

Docker Compose rend l'atelier reproductible.

Message final : une sécurité propre dans les microservices doit être centralisée, standardisée et facile à maintenir.